



Biblioteca de videojuegos

Agustín Borreguero Castro



Índice

Parte A · Análisis del desarrollo móvil	3
Ventajas e inconvenientes.	3
Elección tecnológica	4
Ciclo de vida de una Activity	5
Imagen 1 - Ciclo de vida de una Activity	5
Parte B · Especificación de tu app	6
Idea y público objetivo y 3 objetivos de la app	6
Requisitos funcionales	7
Tabla 1 - Requisitos Funcionales	7
Requisitos no funcionales	7
Tabla 2- Requisitos No Funcionales	8
Casos de uso	8
Diagrama de casos de uso	8
Imagen 2 - Diagrama de Casos de Uso	8
Fichas detalladas de casos de uso	9
Parte C · Entorno de desarrollo	11
Creación del proyecto en Android Studio	11
Imagen 3 - Creación del proyecto	11
Configuración de dos dispositivos virtuales (AVD)	12
Tabla 3 - AVD	12
Imagen 4 - Dispositivo Virtual Móvil	12
Imagen 5 - Dispositivo Virtual Tablet	12
Ejecución de la app en ambos dispositivos	13
Imagen 6 - Pantalla Principal Móvil	13
Imagen 7 - Pantalla Principal Tablet	13
Imagen 8 - Pantalla Secundaria Móvil	14
Imagen 9 - Pantalla Secundaria Tablet	14
Explica brevemente la estructura del proyecto	15
Parte D · Recursos de la aplicación	15
Textos e internacionalización	15
Imagen 10 - Carpetas de textos	15
Imagen 11 - Pantalla Principal en Inglés	16
Imagen 12 - Pantalla Secundaria en Inglés	16
Imágenes	16
Imagen 13 - Imágenes de la aplicación	17
Colores	17
Imagen 14 - Fichero con los colores definidos	17
Estilos y tema personalizado	18
Imagen 15 - Carpetas con los estilos y temas	18



Icono de la aplicación	18
Imagen 16 - Icono al iniciar la aplicación	19
Diseño de las pantallas	19
Imagen 17 - Pantallas de la aplicación	19
Conclusión	19



Parte A · Análisis del desarrollo móvil

Ventajas e inconvenientes.

Una de las principales ventajas de desarrollar aplicaciones para dispositivos móviles es que el dispositivo está siempre con los usuarios y permite utilizar funciones como la pantalla táctil y diferentes sensores. Además, las aplicaciones pueden enviar notificaciones y acceder a funciones del móvil. Por otra parte, una de las desventajas es que los móviles tienen menos recursos que los ordenadores y hay que tener en cuenta varias limitaciones.

- **Batería:** la aplicación no debe estar haciendo tareas innecesarias todo el rato porque consumiría mucha batería. Evitaría tener que hacer procesos innecesarios en segundo plano.
- **Almacenamiento:** el espacio en los móviles es limitado. Los datos de los videojuegos y sus imágenes haría que ocuparan un espacio razonable para no llenar mucho el almacenamiento.
- **Tamaño de pantalla:** existen móviles con distintos tamaños y resoluciones. La interfaz de mi aplicación se adaptaría para que la lista de los videojuegos, botones y textos se vean correctamente.
- **Conectividad:** la conexión puede fallar en cualquier momento. Mi aplicación debería poder acceder a los datos de los videojuegos aunque el usuario no tenga Internet.
- **Fragmentación:** existen muchas versiones de Android y dispositivos. Eligiría una versión mínima y probar la aplicación en distintos dispositivos para evitar problemas de compatibilidad.

Elección tecnológica

Características	Nativo	Multipaltaforma	Híbrido
Lenguajes	Kotlin / Java	Dart, JavaScript, etc.	HTML, CSS y JavaScript
Rendimiento	Muy alto	Casi nativo	Menor que el nativo
Acceso al hardware	Completo	Amplio, normalmente mediante plugins	Mediante plugins
Coste	Mayor si se desarrollan varias versiones	Menor al compartir código	Menor
Ejemplo	Android + Kotlin	Flutter	Ionic + Capacitor

El desarrollo nativo consiste en crear la aplicación únicamente para un sistema operativo, el multiplataforma permite reutilizar el código para distintos sistemas, y el híbrido utiliza tecnologías web para crear la aplicación.

En mi caso voy a utilizar desarrollo nativo Android porque voy a poder aprovechar mucho mejor las características del dispositivo y obtener un buen rendimiento.

Ciclo de vida de una Activity

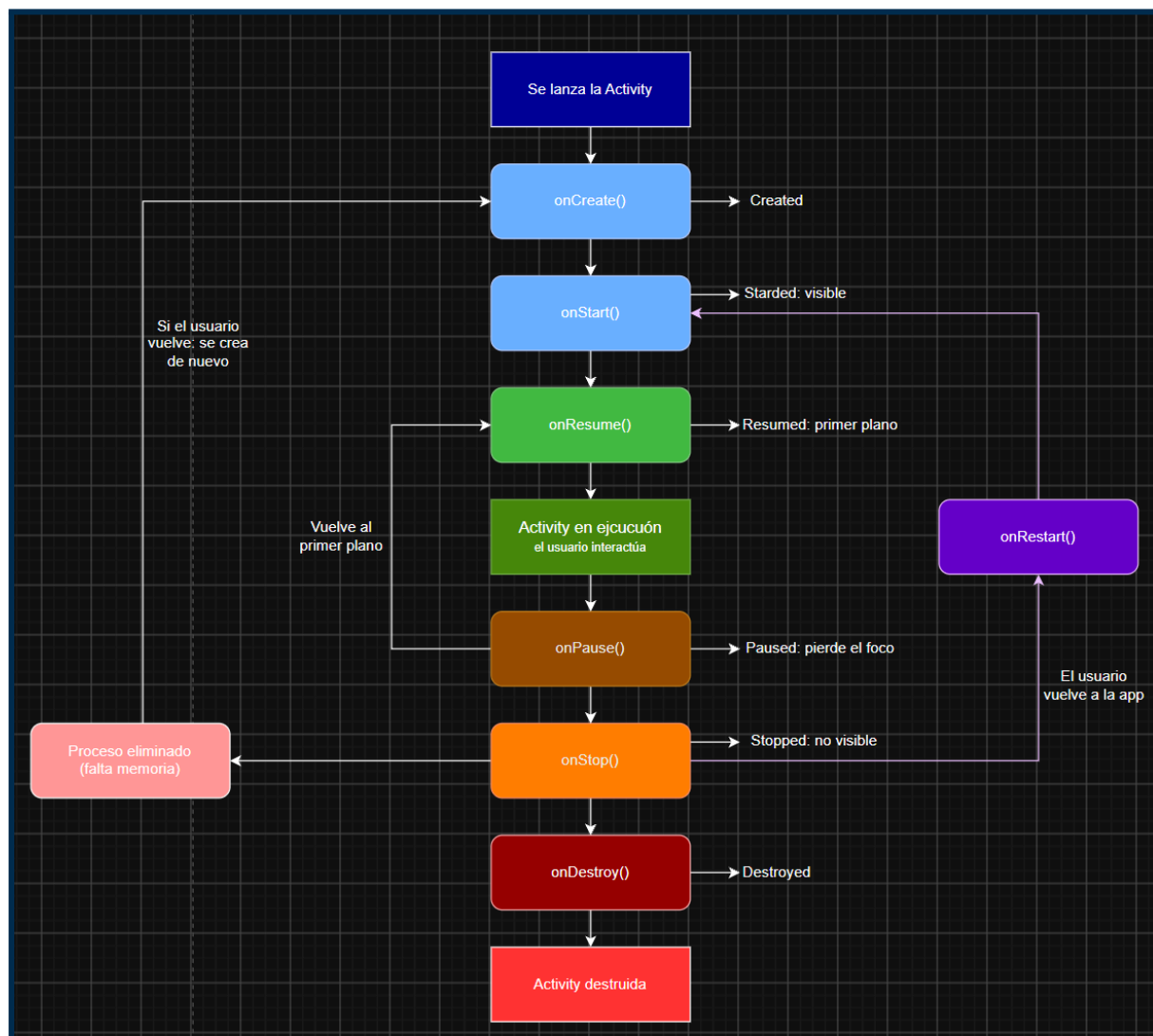


Imagen 1 - Ciclo de vida de una Activity

El usuario recibe una llamada

La llamada pasa a primer plano, por lo que:

onPause() -> onStop()

Cuando el usuario vuelve a la aplicación:

onRestart() -> onStart() -> onResume

En mi app, la biblioteca debería volver al estado en el que estaba antes de la llamada.



El usuario gira el móvil

Al cambiar la orientación, el Activity se recrea.

onPause() -> onStop() -> onDestroy() -> onCreate() -> onStart() -> onResume()

Por ejemplo, si el usuario está viendo los detalles de un videojuego, la pantalla se volvería a crear adaptándose a la nueva orientación.

El usuario pulsa el botón Atrás

onPause() -> onStop() -> onDestroy()

Por ejemplo, si está viendo los detalles de un videojuego y pulsa Atrás, esa pantalla se cerraría y se liberarían sus recursos.

Parte B · Especificación de tu app

Idea y público objetivo y 3 objetivos de la app

La aplicación que voy a desarrollar va a ser una "Biblioteca de Videojuegos" donde el usuario podrá guardar y organizar los videojuegos que tiene o que quiere jugar. Cada juego tendrá información como el nombre, plataforma, género, valoración y estado (jugado o pendiente). También se podrán buscar y filtrar los juegos para encontrarlos más fácilmente. La aplicación está pensada para personas que tienen muchos juegos y quieren organizarlos en un solo sitio. El público objetivo serán jóvenes y adultos aficionados a los videojuegos.

Objetivos de la aplicación

1. **Permitir registrar al menos 25 juegos** con sus datos básicos, sin que la aplicación pierda la información al cerrarse.
2. **Permitir encontrar un videojuego en menos de 4 segundos** utilizando el buscador o los propios filtros de la aplicación.
3. **Permitir modificar el estado y valoración de un videojuego** en un máximo de 3 acciones desde su pestaña de información.



Requisitos funcionales

ID	Requisito Funcional	Prioridad
RF-01	La aplicación permitirá añadir un videojuego con su nombre, plataforma, género y estado.	Alta
RF-02	La aplicación permitirá consultar la lista de videojuegos guardados y acceder a la información de cada uno.	Alta
RF-03	La aplicación permitirá modificar los datos de un videojuego ya guardado anteriormente.	Alta
RF-04	La aplicación permitirá eliminar un juego de la biblioteca.	Media
RF-05	La aplicación permitirá buscar videojuegos por su nombre.	Alta
RF-06	La aplicación permitirá filtrar los juegos por plataforma y género.	Media

Tabla 1 - Requisitos Funcionales

Requisitos no funcionales

ID	Requisito No Funcional	Tipo
RNF-01	La pantalla principal deberá mostrar la lista de videojuegos en menos de 2 segundos después de abrir la aplicación, utilizando una biblioteca de hasta 100 videojuegos.	Rendimiento
RNF-02	Los botones principales deberán ser suficientemente grandes para poder pulsarlos fácilmente y las opciones principales deberán estar disponibles sin realizar más de 3 acciones.	Usabilidad
RNF-03	La aplicación será compatible con API 24 ("Nougat"; Android 7.0), que será la versión mínima de Android utilizada para ejecutar y probar la aplicación. Se ha elegido esta versión porque es la SDK que tengo elegida en Android Studio.	Compatibilidad
RNF-04	La aplicación deberá conservar los datos de los videojuegos después de cerrarla y volver a abrirla.	Fiabilidad

RNF-05	La aplicación no necesitará conexión a Internet para consultar, añadir, modificar o eliminar los videojuegos almacenados localmente.	Disponibilidad
--------	--	----------------

Tabla 2- Requisitos No Funcionales

Casos de uso

La aplicación tendrá un único actor principal:

- **Usuario:** persona que utiliza la aplicación para gestionar su biblioteca de videojuegos.

Diagrama de casos de uso

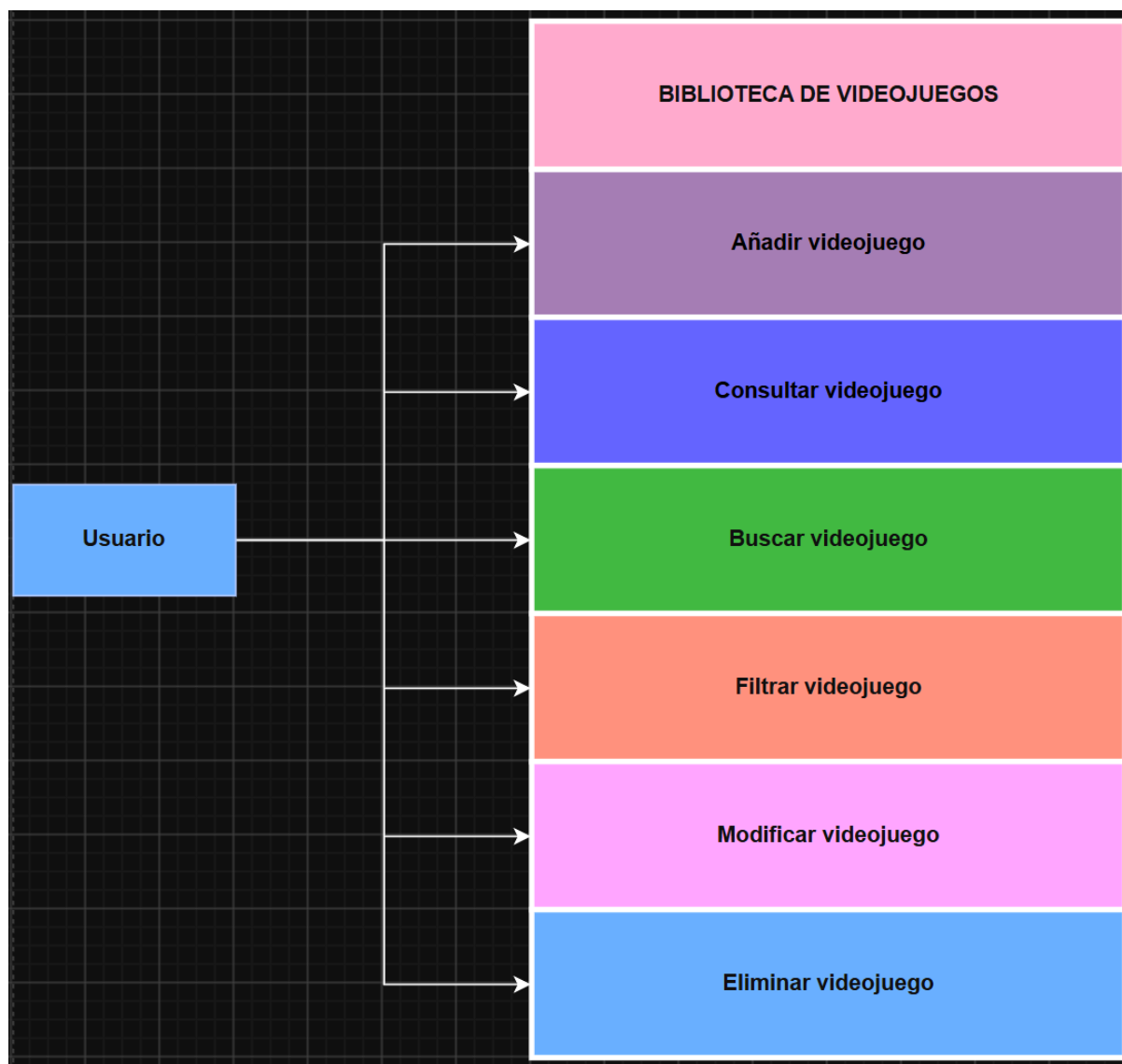


Imagen 2 - Diagrama de Casos de Uso



Los principales casos de uso son:

1. Añadir videojuegos
2. Consultar videojuegos
3. Buscar videojuego
4. Filtrar videojuegos
5. Modificar videojuego
6. Eliminar videojuego

Fichas detalladas de casos de uso

CU-01 · Añadir videojuego

Actor: Usuario.

Precondición: La aplicación está abierta y el usuario se encuentra en la biblioteca de videojuegos.

Flujo principal:

- El usuario pulsa el botón “Añadir videojuego”.
- La aplicación muestra un formulario.
- El usuario introduce el nombre del videojuego.
- Selecciona la plataforma y el género.
- Indica si el videojuego está jugado o pendiente.
- El usuario pulsa “Guardar”.
- La aplicación comprueba que los datos obligatorios están completos.
- La aplicación guarda el videojuego.
- El nuevo videojuego aparece en la biblioteca.

Flujo alternativo:

Si el usuario no introduce alguno de los datos obligatorios, la aplicación muestra un mensaje indicando qué campo falta y no guarda el videojuego.

Si el usuario cancela la operación antes de guardar, vuelve a la biblioteca sin crear ningún videojuego.

Postcondición:

El videojuego queda guardado en la biblioteca y puede consultarse, buscarse, modificarse o eliminarse posteriormente.



CU-02 · Modificar videojuego

Actor: Usuario.

Precondición: Debe existir al menos un videojuego guardado en la biblioteca.

Flujo principal:

- El usuario selecciona un videojuego de la biblioteca.
- La aplicación muestra su información.
- El usuario pulsa "Editar".
- La aplicación muestra los datos actuales.
- El usuario modifica uno o varios datos, por ejemplo el estado o la valoración.
- El usuario pulsa "Guardar cambios".
- La aplicación comprueba los datos introducidos.
- La aplicación guarda los cambios.
- Se muestra la información actualizada del videojuego.

Flujo alternativo:

Si el usuario deja vacío un campo obligatorio, la aplicación muestra un mensaje y no guarda los cambios.

Si el usuario cancela la edición, se mantienen los datos anteriores del videojuego.

Postcondición:

El videojuego queda actualizado con los nuevos datos introducidos por el usuario.



Parte C · Entorno de desarrollo

Creación del proyecto en Android Studio

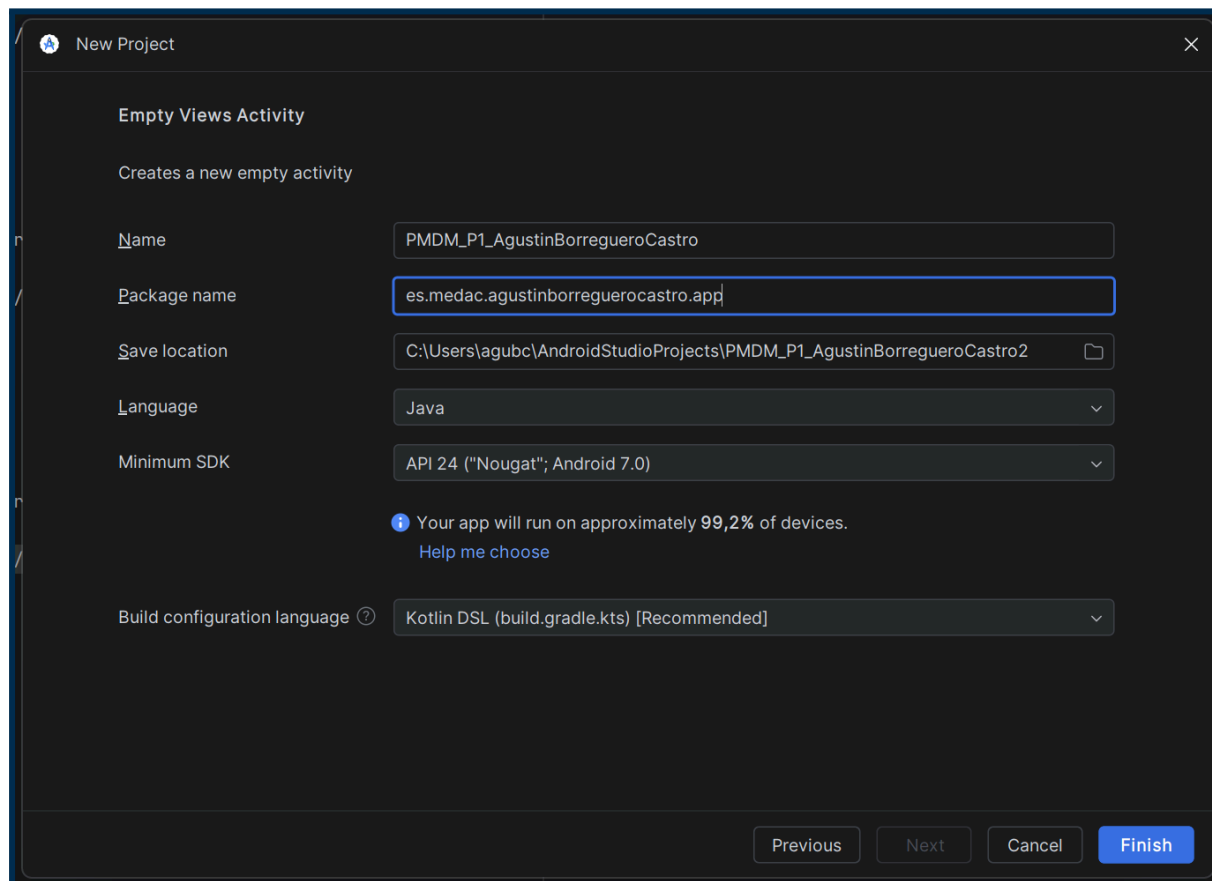


Imagen 3 - Creación del proyecto

Configuración de dos dispositivos virtuales (AVD)

Dispositivo	Tipo	Resolución	Densidad	API
Medium Phone API 37.0	Móvil	1080 × 2400 px	420 dpi	37
Pixel Tablet	Tablet	2560 × 1600 px	320 dpi	35

Tabla 3 - AVD



Imagen 4 - Dispositivo Virtual Móvil

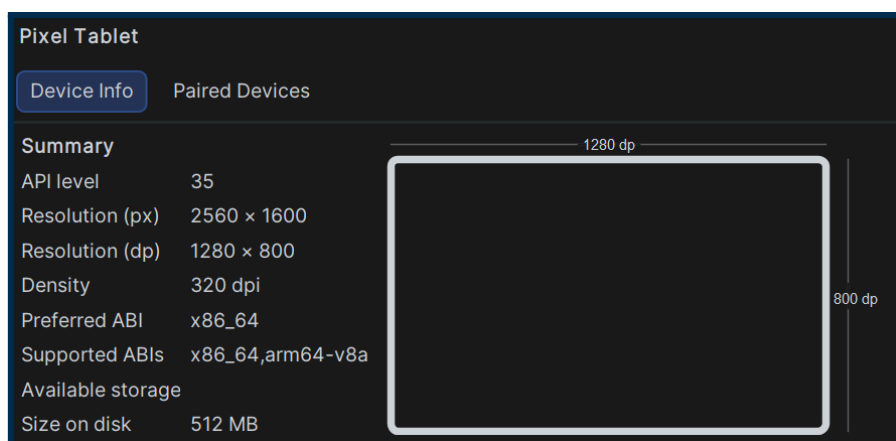


Imagen 5 - Dispositivo Virtual Tablet



Ejecución de la app en ambos dispositivos

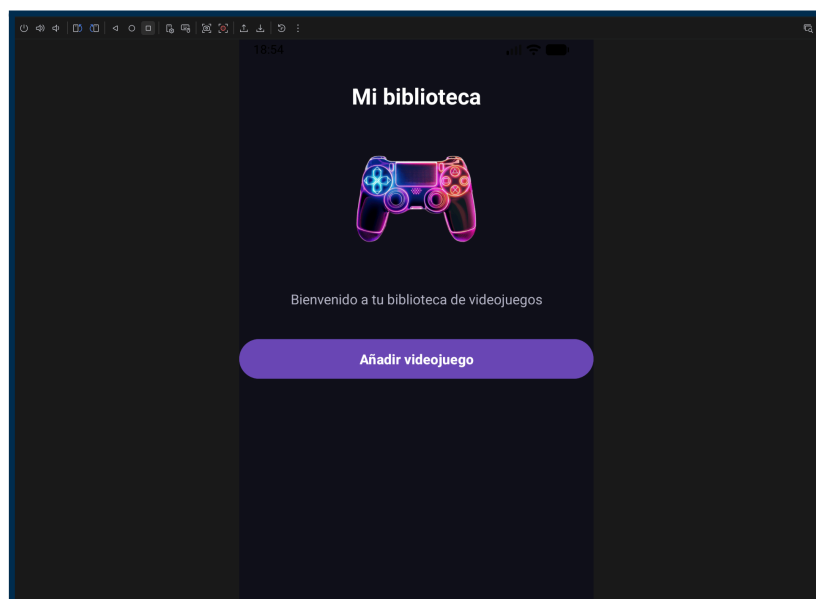


Imagen 6 - Pantalla Principal Móvil

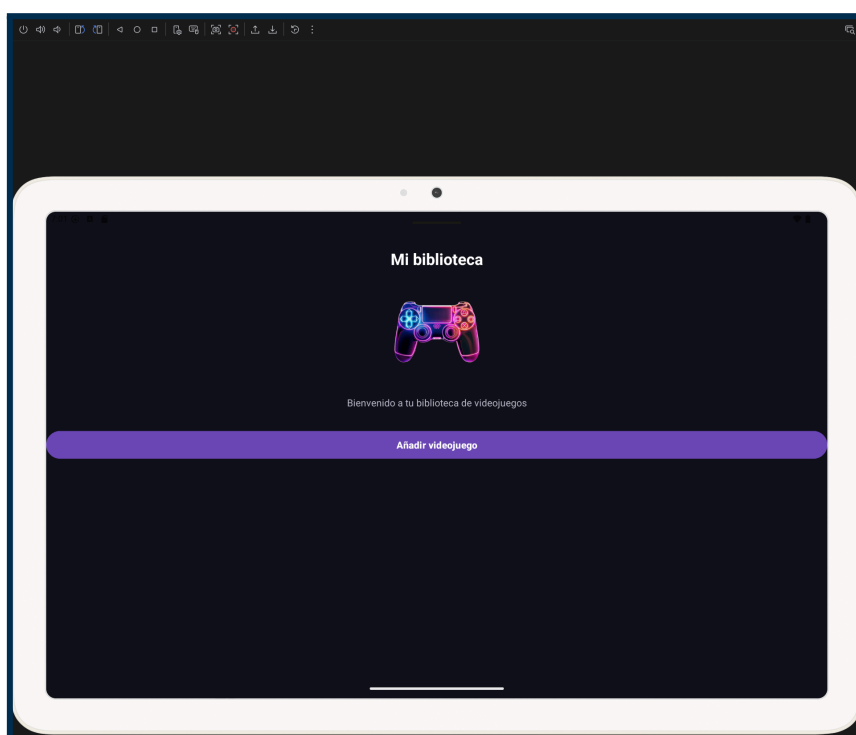


Imagen 7 - Pantalla Principal Tablet



A screenshot of a mobile application interface for adding a video game. The screen is dark-themed. At the top, the title 'Añadir videojuego' is centered. Below it, there are several input fields: 'Nombre' with a placeholder 'Escribe el nombre del videojuego', 'Plataforma' with a dropdown arrow, 'Género' with a dropdown arrow, 'Estado' with a dropdown arrow, and 'Valoración' with five star icons. At the bottom, there are two large buttons: a green one labeled 'Guardar' and a red one labeled 'Cancelar'. The status bar at the top shows the time 19:09 and signal strength.

Imagen 8 - Pantalla Secundaria Móvil

A screenshot of the same 'Añadir videojuego' form, but displayed on a tablet. The layout is identical to the mobile version, with the same fields and buttons. The status bar at the top shows the time 7:04. The tablet's bezels are visible around the screen.

Imagen 9 - Pantalla Secundaria Tablet

Explica brevemente la estructura del proyecto

El proyecto está organizado en varias partes, cada una con una función específica:

- **AndroidManifest.xml:** contiene la información principal de la aplicación, como las actividades que utiliza y la configuración necesaria para ejecutarse.
- **Java:** contiene las clases que controlan las pantallas de la aplicación. En este proyecto están `MainActivity.java`, correspondiente a la pantalla principal, y `AddGameActivity.java`, que muestra el formulario para añadir videojuegos.
- **Res:** contiene los recursos de la aplicación. Dentro de esta carpeta se encuentra `layout`, con los archivos XML que definen el diseño de las pantallas; `drawable`, con las imágenes y los recursos gráficos; `mipmap`, con el icono de la aplicación; y `values`, con los textos, colores, estilos y temas. En `values-en` están los textos traducidos al inglés.
- **Activity y layout:** las `Activities` representan las pantallas y permiten gestionar su funcionamiento, mientras que los `layouts XML` definen su apariencia visual. Por ejemplo, `MainActivity.java` utiliza `activity_main.xml` y `AddGameActivity.java` utiliza `activity_add_game.xml`.
- **Gradle:** se encarga de configurar y construir el proyecto, gestionar las dependencias y definir opciones de compilación. Sus archivos de configuración permiten que Android Studio compile la aplicación correctamente.

Parte D · Recursos de la aplicación

Textos e internacionalización

Para gestionar los textos de la aplicación he utilizado el archivo **strings.xml**, donde he guardado los títulos, botones, mensajes y otros textos que aparecen en las pantallas. De esta forma, no es necesario escribir los textos directamente en los archivos XML.

También he creado el archivo `strings.xml` dentro de la carpeta `values-en` para traducir la aplicación al inglés. Android selecciona automáticamente el idioma correspondiente según

la configuración del dispositivo. Además, he utilizado listas de recursos para mostrar las diferentes plataformas, géneros y estados de los videojuegos.

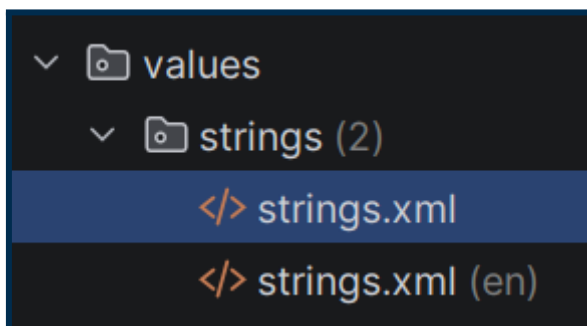


Imagen 10 - Carpetas de textos

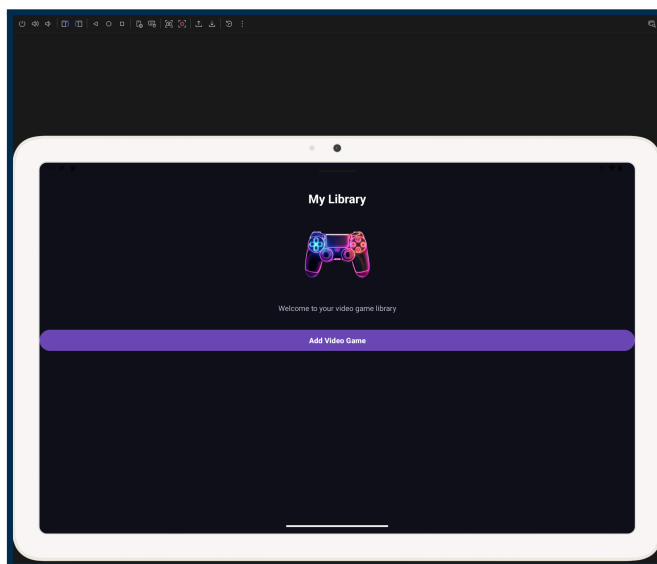


Imagen 11 - Pantalla Principal en Inglés

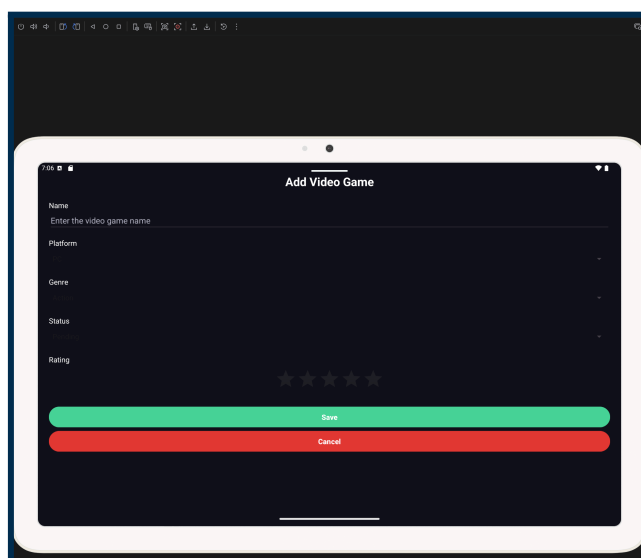


Imagen 12 - Pantalla Secundaria en Inglés

Imágenes

He utilizado dos tipos de imágenes en la aplicación: una imagen vectorial y otra de tipo bitmap.

La imagen vectorial es adecuada para iconos y elementos sencillos, ya que se puede ampliar o reducir sin perder calidad y ocupa poco espacio. En cambio, la imagen bitmap permite mostrar ilustraciones con más detalles y variedad de colores, aunque su tamaño depende de la resolución y del formato utilizado.

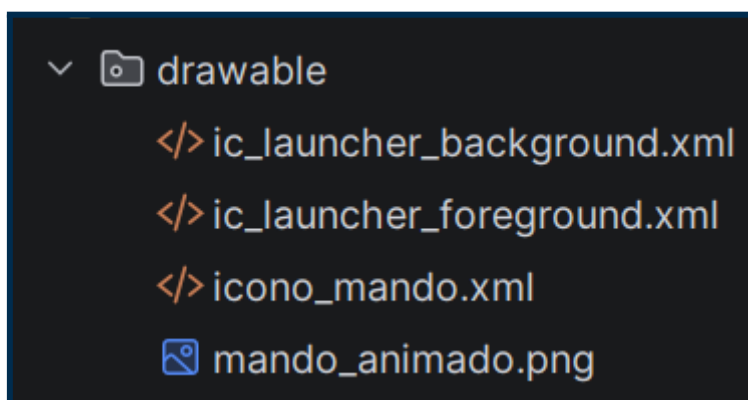


Imagen 13 - Imágenes de la aplicación

Colores

He creado una paleta de colores propia en el archivo **colors.xml**, utilizando nombres que indican la función de cada color, como **color_primario**, **color_fondo**, **color_texto** y **color_acento**.

He elegido una combinación de tonos oscuros con detalles morados y azules para darle a la aplicación un estilo relacionado con los videojuegos. También he definido un color verde para el botón de guardar y uno rojo para el botón de cancelar.

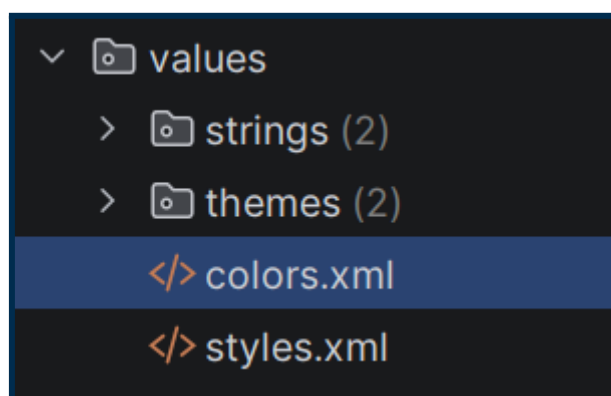


Imagen 14 - Fichero con los colores definidos

Estilos y tema personalizado

He creado cuatro estilos propios en el archivo **styles.xml**: **TextoTitulo**, **BotonPrincipal**, **BotonGuardar** y **BotonCancelar**. El primero permite mantener el mismo formato en los

títulos, mientras que el segundo define la apariencia de los botones principales, el tercero la apariencia del botón de guardar y el cuarto la apariencia del botón de cancelar.

También he personalizado el tema de la aplicación en `themes.xml`, utilizando los colores definidos anteriormente para adaptar el aspecto general de la interfaz.

Para establecer las medidas he utilizado **dp** en los tamaños y espacios de los elementos, y **sp** en los tamaños de texto, facilitando que la interfaz se adapte a diferentes pantallas y configuraciones de texto.

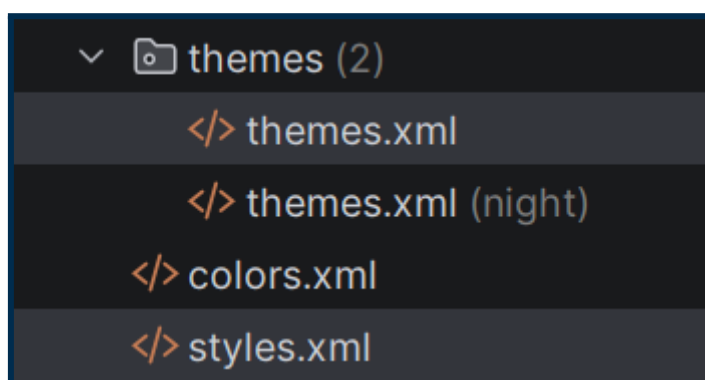


Imagen 15 - Carpetas con los estilos y temas

Icono de la aplicación

He diseñado un icono personalizado para la aplicación utilizando la herramienta **Image Asset** de Android Studio. He elegido un símbolo relacionado con los videojuegos y un fondo morado para mantener la coherencia con los colores de la interfaz.

Android Studio genera los recursos del icono en las carpetas `mipmap`, con diferentes tamaños para que se muestre correctamente en los dispositivos. Después, he configurado el icono en el archivo `AndroidManifest.xml` para que se utilice como icono de la aplicación.

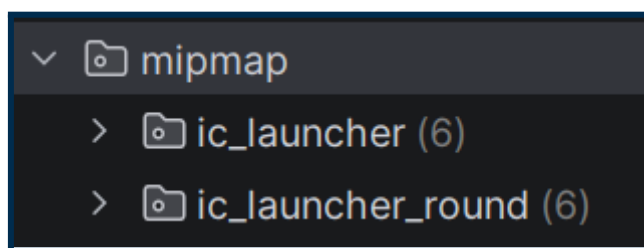


Imagen 16 - Icono al iniciar la aplicación

Diseño de las pantallas

He diseñado dos pantallas para la aplicación "Biblioteca de Videojuegos". La primera es la pantalla principal, que incluye el título de la biblioteca, una ilustración y un botón para añadir videojuegos. La segunda es un formulario para introducir los datos de un videojuego, como el nombre, la plataforma, el género, el estado y la valoración.

Las dos pantallas mantienen la misma paleta de colores y los mismos estilos para conseguir una apariencia uniforme.

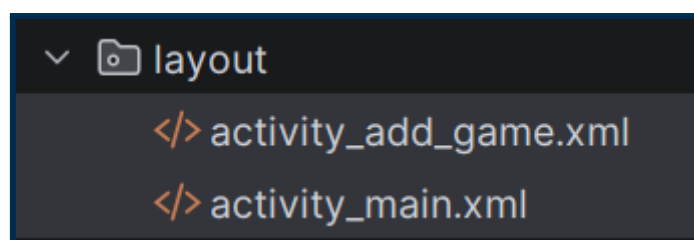


Imagen 17 - Pantallas de la aplicación

Conclusión

Con esta práctica he aprendido a crear una aplicación Android utilizando Android Studio, diseñar pantallas con XML y organizar los recursos, como los textos, colores, imágenes y estilos. También he aprendido a configurar un tema personalizado y a traducir la aplicación al inglés.

Una de las dificultades que he tenido ha sido configurar correctamente los recursos y el idioma de la aplicación para que se mostrara en español y en inglés. Además, he tenido que familiarizarme con la estructura del proyecto y con la creación de las diferentes pantallas.



Como mejora futura, me gustaría implementar filtros para buscar videojuegos según diferentes criterios, ya que no los he añadido en esta práctica. Esto facilitaría encontrar videojuegos concretos y haría que la aplicación fuera más completa y útil.